

架构辩论赛记录

团队：第67组 日期：2026/5/2

1.Prompt与AI回答记录

1.1 成员 A：直接问"推荐什么架构"

Prompt

现在我要开发一个Web应用，为我推荐一个架构。

AI回答

****推荐的Web应用架构：整洁分层架构 + 后端为核****

架构风格

客户端层：Web端 / 移动App / 第三方客户端
| API (REST/GraphQL/gRPC)
▼
网关层：负载均衡 + API网关 + 限流/认证
|
▼
应用层：
|— BFF/API服务：Controller → UseCase → Service → Repository接口
|— 后台服务：任务队列Worker / 定时任务 / 消息消费者
|
▼
基础设施层：PostgreSQL | Redis | MQ | OSS | ES/日志

推荐技术栈

前端

- ****框架****: React + TypeScript 或 Vue3 + TypeScript
- ****状态管理****: Zustand (React) / Pinia (Vue)
- ****UI组件库****: Ant Design / Element Plus / shadcn/ui
- ****构建工具****: Vite
- ****HTTP客户端****: TanStack Query + Axios
- ****路由****: React Router / Vue Router

后端

- ****运行时****: Node.js (推荐) 或 Go
- ****框架****: NestJS (TypeScript首选) / Express + TypeScript
- ****数据库****: PostgreSQL (主库) + Redis (缓存)
- ****ORM****: Prisma 或 TypeORM
- ****API风格****: RESTful + 部分GraphQL (按需)

部署运维

- ****容器化****: Docker + Docker Compose
- ****反向代理****: Nginx
- ****CI/CD****: GitHub Actions / GitLab CI
- ****云平台****: AWS/阿里云/Vercel + Supabase (BaaS选项)

项目结构示例

```
my-web-app/
├── frontend/                # React/Vue前端
│   ├── src/
│   │   ├── components/
│   │   ├── pages/
│   │   ├── hooks/
│   │   ├── services/
│   │   └── utils/
│   └── backend/            # NestJS/Express后端
│       ├── src/
│       │   ├── modules/
│       │   ├── auth/
│       │   ├── users/
│       │   └── common/
│       └── docker-compose.yml
├── nginx.conf
```

1.2 成员 B：提供详细项目约束后问

Prompt

我正在为一个高校校园互助平台 **CampusHub** 设计系统架构，请你担任技术顾问，基于以下详细约束条件，给出具体的架构风格建议（如 MVC + 分层、微服务、事件驱动等）与理由。

项目背景：

平台提供学生之间的互助任务撮合，支持发布需求、接单、订单状态流转、双向评价、基础论坛发帖。

目标用户为在校学生，移动端使用为主（优先适配微信内置浏览器）。

项目定位为 MVP（最小可行产品），核心闭环是“发单 → 接单 → 确认完成 → 互评”。

团队约束：

4 名学生开发者，非全职，总体软件架构经验尚浅。

项目周期约束：

总开发时间仅 10 周，必须交付可演示的核心功能。

非功能约束：

需支持 500 人同时在线，核心操作响应低于 2 秒。

隐私保护要求高：不强制实名，支持匿名发布与匿名评价。

需要内容审核（敏感词过滤）与简单的信用评分机制。

功能优先级：

必须做：注册登录、任务发布/浏览、接单、订单状态管理。

应该做：匿名、评价与信用、基础论坛、站内通知。

不做：即时通讯、智能推荐、复杂信用模型。

我的问题：

综合考虑以上所有条件，请给出一个架构方案并提供建议，解释原因

AI回答

一、推荐架构方案

总体风格：

整体上是一个“一个代码仓库 + 一次部署”的单体应用，内部严格按 表现层（Controller）→ 业务逻辑层

（Service）→ 数据访问层（Repository） 分层设计，即经典的 MVC + 分层架构。

前后端完全分离，后端提供 RESTful API，前端是专门适配微信内置浏览器的 H5 移动端应用，两者独立

开发、独立构建，通过 HTTP 通信。

技术组件参考：

前端：Vue 或 React + Vant/Mint-UI 等移动端组件库，打包成纯 H5，优先保证微信内体验。

后端：可根据团队已有经验选择轻量框架，如 Python Flask/FastAPI 或 Node.js Express，简洁且学习

成本低。

数据库：PostgreSQL 或 MySQL。

缓存：Redis，用于储存用户会话、热点数据（如首页任务列表）以降低响应时间。

内部异步：利用框架自身的异步/事件机制（如 Spring 的 @Async 配合事件监听、Node.js 的 EventEmitter、或 Python 的 asyncio/任务队列），仅用于发送站内通知、更新信用分等非关键路径操作，不用引入独立的消息中间件。

功能与模块拆分（单体内部模块化）：

text

campus-hub

- |— auth（登录注册/微信授权）
- |— task（任务发布、浏览、接单）
- |— order（订单状态流转 发单-接单-完成）
- |— review（双向评价，支持匿名）
- |— credit（简单信用评分）
- |— forum（基础论坛发帖/浏览）
- |— notification（站内通知，异步发送）

所有模块共享同一数据库，通过服务层调用彼此，不引入远程调用。

二、为什么这是最合适的架构风格

1. 适配团队与时间约束：降低复杂度，确保交付

四个非全职、经验尚浅的学生开发者在 10 周内交付可演示的 MVP，首要目标是 降低认知负荷。

单体分层架构是他们最可能熟悉的模式（课堂项目常用），无需学习服务发现、分布式事务、容器编排等复杂概念。

前后端分离既能让前端同学专注移动端适配，又让后端同学聚焦业务逻辑与接口，互不阻塞。

模块按功能垂直划分，四人可分别认领订单、评价、论坛等模块并行开发，代码仍在同一仓库中，集成冲突少。

2. 充分满足非功能需求，不需要微服务

500 人在线、核心操作 <2 秒：对于一个优化良好的单体应用毫无压力。通过连接池、合理索引、

Redis

缓存热点任务列表，单机（如 2 核 4G）即可稳定支撑。

隐私与匿名：是业务逻辑与数据库字段设计的范畴，而非架构风格问题。在任务和评价实体中增加 is_anonymous 字段，Service 层根据其值决定是否隐藏真实身份即可，分层架构天然支持。

敏感词过滤：在任务发布、帖子、评价的 Controller 或 Service 中同步调用过滤库即可，响应依旧在毫

秒级，无需异步。

简单信用评分：在“确认完成”服务中同步计算（如完成数+1，累计评分取均值），同样不需要复杂的事件流。

站内通知：这是唯一可从“事件驱动”思维中受益的场景。当订单状态变更时，可以发布内部事件（如 `OrderCompletedEvent` ），由通知模块异步监听并生成站内消息。这只需利用语言/框架内置的异步能力，不引入外部队列，既解耦了通知逻辑，又不会显著增加架构复杂性。

3. 完美匹配功能优先级

必须做：注册登录、任务 CRUD、接单、状态流转，都对应简单的 REST 接口 → Service → 数据库链路，分层架构直接支持。

应该做：匿名、评价信用、论坛、通知，均可作为新模块插入分层结构中，只依赖已有的 Service 接口。

不做：即时通讯、智能推荐等，避免了我们因过度设计而采用 WebSocket、微服务或复杂推荐引擎的理由。

单体能满足所有需要做的功能。

4. 为未来演进留有余地

严格的分层与模块内聚，使得未来如果需要将“论坛”或“评价”拆成独立微服务时，边界已清晰。但现阶段

绝不这样做，因为 MVP 是为验证想法，而非构建伸缩平台。

若用户量日后真的暴涨，可在该升级点再引入微服务，而现在的架构已成为最好的“模块化单体”起步。

三、不建议采用的架构风格与理由

微服务：

会立即带来服务拆分粒度争论、分布式通信、事务处理、独立部署、监控等巨大开销，4 人新手团队在 10 周内必定无法完成，且 500 在线根本不需要。过早微服务是 MVP 的最大杀手。

完全的事件驱动架构（如 CQRS + Event Sourcing）：

需要独立的消息代理、事件存储、最终一致性处理，极度复杂。简单信用评分、通知不需要这种重型机制，同步处理加少量内部异步已足够清晰。

Serverless：

虽然部署简单，但大量远程调用会带来冷启动延迟，难以保证 2 秒响应；调试、并发控制对经验不丰富的

团队也是挑战，且与微信 H5 的交互没有明显优势。单体部署在一台云服务器上更可控。

四、补充的工程建议（不是架构风格，但助力成功）

1. 技术栈推荐：如果团队对 JavaScript 熟悉，用 Node.js (Express) + Vue 全栈，一门语言到底，学习成

本最低；如果偏好 Python，用 FastAPI + Vue，自动生成 API 文档，利于前后端协作。

2. 部署方案：一台云服务器，使用 Nginx 反向代理前端静态资源与后端 API，挂上域名，用 PM2 或

systemd 管理进程。10 周内完全够用。

3. 微信适配：采用微信网页授权获取 OpenID 作为静默登录凭证，符合“不强制实名”要求；UI 使用响应式

设计，在微信内置浏览器中做好字体、按钮区域适配。

4. 开发节奏：建议第 1-4 周完成登录、任务发布浏览接单的基础闭环（不含匿名与评价）；第 5-7 周补齐

匿名、评价信用与论坛；第 8-10 周全面调试、优化性能和内容过滤，保证可演示。

5. 内容审核：直接集成如 text-filter 或开源敏感词库，在 Service 层调用。若担心性能，只对发布内容

做检查（写操作远少于读操作），完全同步执行，响应时间足够。

总结：一个设计干净、严格分层的 RESTful 单体，配合前端 H5，是你在有限时间、有限人力下完成 CampusHub MVP 的最优架构。它务实、易协作、能快速上线验证核心闭环，同时完全满足性能与隐私需

求，等你拿到反馈后再优化不迟

1.3 成员 C：要求 AI 对比多种方案的优劣

prompt

我正在为高校校园互助平台 **CampusHub** 设计系统架构。请从开发复杂度、可维护性、可扩展性、性能、团队学习成本、10 周内可行性六个维度，对以下架构方案进行比较，并给出推荐结论：1) 前后端分离 + 单体分层架构；2) 微服务架构。项目背景如下：4 人团队，周期 10 周，现有技术基础为 **Vue** 前端 + **Spring Boot** 后端，核心功能包括用户管理、需求发布与浏览、接单与订单流转、评价信用、消息通知。请说明每个方案的优缺点，以及最终推荐理由。

AI回答

微服务架构在长期扩展性、独立部署和复杂业务演进方面更有优势。微服务会显著增加服务拆分、接口治理、部署运维、联调测试和团队协作成本。对于 4 人团队、10 周周期的课程项目，前后端分离 + 单体分层架构更利于控制复杂度与交付风险。

在当前阶段，单体分层方案更适合保证进度、稳定性和实现完整度。应该将 **CampusHub** 当前阶段的系统架构确定为“前后端分离 + 单体分层架构”。

- 当前团队规模较小，微服务会额外引入注册发现、网关、服务拆分、部署监控等工程负担。
- 项目周期只有 10 周，课程项目的核心目标是按时完成可运行、可演示、可说明的系统。
- 现有技术基础已经明确为 **Vue** 前端与 **Spring Boot** 后端，采用单体分层更符合团队已有经验。
- **CampusHub** 目前的业务复杂度尚未达到必须拆分微服务的程度，单体架构足以支撑当前需求。

为什么不选微服务

- 当前系统规模较小，微服务的扩展性优势暂时难以转化为实际收益。
- 微服务会显著提高开发、联调、部署和排错成本。
- 课程作业更强调工程判断与可落地性，不需要为了追求“先进架构”而过度设计

1.4 成员 D：让 AI 扮演架构师角色进行分析

prompt

请扮演资深软件架构师，参与**CampusHub**校园互助平台架构辩论赛，围绕“非核心流程（通知、信用分更新）的异步处理方案”展开分析，项目约束为4人学生团队、10周交付MVP、500人在线、核心接口响应<2秒，对比“进程内轻量异步事件机制”与“外部消息队列（RabbitMQ/Kafka）”两种方案，从开发复杂度维度分析，给出支持“进程内轻量异步事件机制”的观点，贴合学生团队实际。

AI回答

从开发复杂度来看，进程内异步基于后端框架（**Python Flask**）内置功能实现，无需额外学习、部署外部组件，复杂度极低，4人学生团队可快速上手；而外部消息队列需专门学习**RabbitMQ/Kafka**的部署、配置、调试，复杂度远超学生团队的能力范围，不利于快速推进项目。

prompt

请扮演资深软件架构师，继续从团队适配性维度，补充支持“进程内轻量异步事件机制”的观点，结合学生团队的特点分析。

AI回答

从团队适配性来看，进程内异步无需团队掌握分布式、消息队列专业知识，贴合学生团队架构经验有限的特点，4人团队可快速熟悉并应用；而外部消息队列要求团队具备分布式开发、组件运维等经验，学生团队缺乏相关积累，难以胜任，会严重影响开发进度。

prompt

请扮演资深软件架构师，结合10周交付MVP的约束，从交付周期维度分析，为什么不推荐外部消息队列？

AI回答

从交付周期来看，进程内异步开发、调试流程简单，可快速集成到现有架构中，无需额外花费时间搭建基础组件，不影响10周MVP交付目标；而外部消息队列仅基础组件搭建、环境配置就需2-3周，还需花费时间调试适配，极易导致项目延期，不符合短周期交付要求。

prompt

请扮演资深软件架构师，针对500人在线、核心接口响应<2秒的性能要求，两种方案的表现有什么差异？

AI回答

从性能表现来看，进程内异步无网络调用开销，可避免额外延迟，能有效保障核心接口响应速度，完全满足500人在线、响应<2秒的要求；而外部消息队列存在网络传输延迟，不仅会增加架构复杂度，还可能拖慢核心接口响应，反而影响用户体验，不符合项目性能约束。

prompt

请扮演资深软件架构师，从运维成本角度，分析哪种方案更适合学生团队？

AI回答

从运维成本来看，进程内异步无需额外部署、监控外部组件，调试、排错简单，4人学生团队可轻松维护，无需投入额外精力；而外部消息队列需专门部署、监控组件，还要处理消息丢失、组件故障等问题，运维负担极大，超出学生团队的运维能力，会增加项目风险。

prompt

请扮演资深软件架构师，反方提出“进程内异步无持久化，应用重启会丢失通知，存在风险”，请结合项目场景，给出反驳观点。

AI回答

首先，我们的核心业务是“发单-接单-完成-互评”，通知、信用分更新属于非核心流程，即便少量丢失，也不会影响核心业务闭环，不会对项目造成实质性影响；其次，校园平台峰值仅500人在线，事件量极小，应用重启的概率本身较低，丢失通知的概率更是微乎其微，无需过度追求持久化；若为了这点非核心风险引入外部消息队列，会大幅增加架构复杂度和开发成本，得不偿失。

prompt

请扮演资深软件架构师，反方又质疑“进程内异步扩展性差，未来用户量增加无法支撑”，请结合项目定位和学生团队情况，反驳该观点。

AI回答

当前项目的核心目标是10周交付MVP，定位是校内小范围使用，未来短期内用户量大幅激增的可能性极低，无需提前考虑分布式扩展需求；即便未来需要扩展，我们的架构采用模块化设计，可基于现有模块平滑调整异步机制，无需推倒重构核心代码；而反方推荐的外部消息队列，是为分布式场景设计的，当前引入属于过度设计，既增加了开发和运维成本，又超出了学生团队的能力范围，完全不符合项目实际。

prompt

请扮演资深软件架构师，总结一下，结合所有维度，为什么进程内轻量异步事件机制是当前项目的最优选择？

AI回答

综合开发复杂度、团队适配性、交付周期、性能、运维成本五个维度来看，进程内轻量异步事件机制完全贴合4人学生团队、10周交付MVP的核心约束，既满足非核心流程异步处理的需求，又避免了外部消息队列的过度设计；它复杂度低、易上手、运维简单，能确保项目按时交付，同时满足性能要求，是当前项目场景下最务实、最可落地的最优选择。

2. 团队辩论会最终方案选择

选择架构

前后端分离的分层单体架构

架构全景图

```
[前端层] Vue.js
  (单页面应用，负责界面展示与用户交互)
↓ HTTPS / RESTful API
[接入层] Nginx
  (托管Vue静态资源 + 反向代理后端API + 解决跨域 + 负载均衡)
↓
[应用层] Spring Boot 单体后端 (分层架构)
├─ Controller (表示层)：接收请求、参数校验、返回JSON
├─ Service (业务逻辑层)：订单流转、信用计算、敏感词过滤
├─ Repository (数据访问层)：封装MySQL操作
└─ Security (安全层)：Spring Security + JWT / Session认证
↓
[数据层] MySQL
  (主数据库，存储用户、订单、需求等核心业务数据)
```

架构选择原因总结

我们最终选择前后端分离的分层单体架构，核心考量是学生团队的技术适配性与10周交付可行性。其中前端采用 Vue.js，因其学习门槛低、单文件组件开发效率高、中文生态完善，即使没有学过前端的同学也能比较快速地上手；后端采用 Spring Boot，利用其配置简单、IoC容器自动管理对象、生态成熟稳定的特点，能够快速构建可靠的事务处理能力，尤其适合校园场景下订单流转、信用计算等业务逻辑；数据库选用 MySQL，与 Spring Boot生态无缝融合，读写性能优秀且经受过大量真实场景考验。在安全方面，Spring Security 配合 CorsFilter 和 UsernamePasswordAuthenticationFilter，实现了认证与跨域策略的统一集中管理，避免自行实现安全组件带来的漏洞风险。部署上采用 Nginx 托管前端静态资源并对后端API进行反向代理，不仅解决了跨域问题、隐藏后端细节，还为未来横向扩展（多实例负载均衡）预留了空间。整体架构虽为单体，但通过严格的分层设计

(Controller-Service-Repository)，保证了代码可维护性和未来模块化拆分的能力，既降低了学生团队的开发复杂度，又确保了在10周内能够稳定交付一个安全、可靠、可演进的校园交易平台。